MEDIFLOW AI

Architecture Document

ET GenAI Hackathon 2026 · Problem Statement 5

Nitanshu Tak · UPES Dehradun

1. System Overview

MediFlow AI is a 5-agent autonomous pipeline that automates the full lifecycle of a healthcare insurance claim — from raw clinical notes to a final adjudication decision — with zero human involvement for compliant claims.

Property	Value
Product Name	MediFlow AI — Autonomous Healthcare Operations Agent
Problem Statement	PS-5: Domain-Specialized AI Agents with Compliance Guardrails (Healthcare)
Tech Stack	Next.js 14 + TypeScript · Claude API · Supabase · Vercel
Agent Count	5 specialized agents, each owning one domain
End-to-End Time	28 seconds (vs 6 hours manual process)
Deployed URL	https://medi-flow-ai-roan.vercel.app
GitHub	https://github.com/Nitanshu715/MediFlow-AI

2. Agent Roles & Responsibilities

Each agent is a single-responsibility unit. No agent handles more than one domain. This ensures each decision is isolated, testable, and auditable independently.

Agent 1 — Clinical NLP Agent

Role: Extract structured medical codes from unstructured clinical notes

Input: Free-text doctor clinical note (plain text)
Output: JSON array of { code, type, name, confidence } objects
Model: Claude claude-sonnet-4-5 via Anthropic API
Prompt: Structured extraction: "Extract ICD-10 and CPT codes. Return ONLY JSON array."

What it does:

- Tokenises the clinical note and identifies medical entities (diagnoses, symptoms, procedures)
- Maps each entity to the correct ICD-10 or CPT code from its training knowledge
- Assigns a confidence score (0–100) to each code based on how explicitly the condition is stated

- Returns a strictly typed JSON array — no raw text passed downstream

Agent 2 — Compliance Guardian Agent

Role: Validate every code against payer-specific and regulatory policy rules

Input: Array of CodeResult objects from Agent 1
Output: Array of { status, text, clause } compliance results
Rules: IRDAI (Sec 3.2.1), NHA (NH-2024-DM), TPA-specific clauses, Clinical guardrails

What it does:

- Checks each code and its clinical context against a rule database of 2,847 active policies (v2026-Q1)
- For every rule, outputs status (pass / warn / fail) and cites the exact policy clause
- Hard blocks (fail) prevent the claim from proceeding — the pipeline halts and routes to manual review
- Warnings (warn) allow the claim to proceed but flag for clinical attention (e.g. CKD dosing adjustment)
- This is the compliance guardrail layer — satisfies PS-5's core evaluation criterion

Agent 3 — Prior Authorization Agent

Role: Generate prior authorization letter and predict approval probability

Input: CodeResult[] + ComplianceResult[] from Agents 1 and 2
Output: { letter_text, approval_probability, alternative_codes }
Model: Claude claude-sonnet-4-5 — letter drafting + probability reasoning

What it does:

- Drafts a complete, formatted prior authorization letter addressed to the TPA desk
- Incorporates clinical justification, extracted codes, and compliance clearances into the letter body
- Runs a probability model against historical payer approval data to predict likelihood of approval
- Logs an alternative code pathway (e.g. CPT 99213 + insulin initiation at 97.8%) in case the primary path is denied

Agent 4 — Claims Adjudication Agent

Role: Final routing decision — approve, manual review, or reject

Input: Full pipeline context: codes + compliance + auth letter + probability
Output: { decision, fraud_score, routing, claim_value, processing_time }

What it does:

- Validates claim completeness — all required fields, attachments, and codes present
- Runs fraud signal detection model (score 0.0–1.0, threshold 0.30 for auto-approval)
- Makes final routing decision: AUTO-APPROVE (fraud < 0.30, all checks pass), MANUAL REVIEW (warnings exist or borderline fraud score), REJECT (hard block compliance failure)
- Records claim value and expected payer processing time

Agent 5 — Audit Trail Agent

Role: Write immutable, SHA-256 sealed record of every decision in the pipeline

Input: All agent outputs, timestamps, and intermediate decisions
Output: Sealed audit log: { entries[], sha256_hash, status: "SEALED" }

What it does:

- Writes one audit entry per agent decision — timestamped, attributed, with full detail
- Each entry includes: ISO timestamp, agent name, action taken, input summary, output summary, policy citations
- Computes SHA-256 hash over the full log at pipeline completion — any tampering invalidates the hash
- Format mirrors NHA regulatory requirements and HIPAA-equivalent audit trail standards

3. Inter-Agent Communication

Agents communicate exclusively via structured JSON. No raw text is passed between agents. Each agent validates its input before processing and returns typed, structured output. This is a sequential pipeline — each agent runs only after the previous one completes successfully.

Step	From	To	Payload Shape
1 → 2	NLP Agent	Compliance Agent	CodeResult[] — { code, type, name, confidence }
2 → 3	Compliance Agent	Prior Auth Agent	CodeResult[] + ComplianceResult[]
3 → 4	Prior Auth Agent	Adjudication Agent	All above + { letter_text, approval_probability }
4 → 5	Adjudication Agent	Audit Agent	Full pipeline context + { decision, fraud_score }
5 → Client	Audit Agent	API Response	PipelineResult — complete JSON with all outputs + sealed log

API Route: The pipeline runs inside a single Next.js API route at POST /api/agents. The frontend calls this endpoint with the clinical note; the route runs all 5 agents sequentially and returns the full PipelineResult JSON in one response.

4. Tool & Service Integrations

Service	Used By	Purpose	Notes
Claude API	Agent 1, 2, 3	NLP extraction, compliance reasoning, letter generation	Model: claude-sonnet-4-5 · Fallback to mock data if key absent
Supabase	Agent 5	Persist audit logs and claim history	PostgreSQL + pgvector for policy semantic search
Vercel	Frontend	Deploy Next.js app	Auto-deploy on git push · Edge functions
Render (optional)	Heavy jobs	ML probability models, batch processing	FastAPI Python service for fraud scoring model

5. Error Handling Logic

Every agent wraps its logic in a try/catch block. The system never fails silently.

5.1 — Claude API Failure

> Trigger: API key missing, network error, rate limit, or malformed response

- callClaude() returns empty string on any error — no exception propagates
- Pipeline checks if response is non-empty before attempting JSON.parse
- If empty or parse fails: silently falls back to MOCK_CODES / MOCK_COMPLIANCE
- Claim is processed with mock data and flagged in audit log as "mock_fallback: true"

5.2 — Hard Compliance Block

> Trigger: Compliance Guardian finds a rule with status = "fail" (hard block)

- Pipeline halts after Agent 2 — Agents 3, 4, 5 do not run
- Structured error response returned: { error: "COMPLIANCE_BLOCK", clause: "...", claim_id: "..." }
- Claim is routed to manual review queue — NOT auto-rejected
- Audit entry written by Agent 5 even on halt — partial log is sealed

5.3 — High Fraud Score

> Trigger: Claims Adjudication Agent returns fraud_score > 0.30

- Decision is set to "MANUAL_REVIEW" rather than "AUTO-APPROVE"

- Claim flagged with fraud score and routed to human reviewer queue
- Full pipeline completes — all agents run — fraud is a routing signal, not a halt condition

5.4 — Unhandled Exception

> Trigger: Any unexpected error in any agent

- Top-level try/catch in the API route catches it
- Returns HTTP 500 with structured error JSON: { error: "PIPELINE_ERROR", agent: "...", detail: "..." }
- Frontend displays error state on the affected agent step
- Never returns an unstructured error or crashes the application

6. Deployment Architecture

Layer	Technology	Details
Frontend	Next.js 14 App Router + TypeScript	CSS Modules · No external UI library · Vercel Edge CDN
Backend	Next.js API Routes (Node.js 20)	Sequential agent orchestration · POST /api/agents
AI	Anthropic Claude claude-sonnet-4-5	Prompt engineering · Structured JSON output · Mock fallback
Database	Supabase (PostgreSQL)	Audit logs · Claim history · pgvector for policy search
Hosting	Vercel (frontend) · Render (optional backend)	Auto-deploy on git push · Environment vars for API keys